

好。我们现在正式以你之前那份设计书作为 **Mini-DFT v0.1** 的唯一架构基准，不再重新改目录。

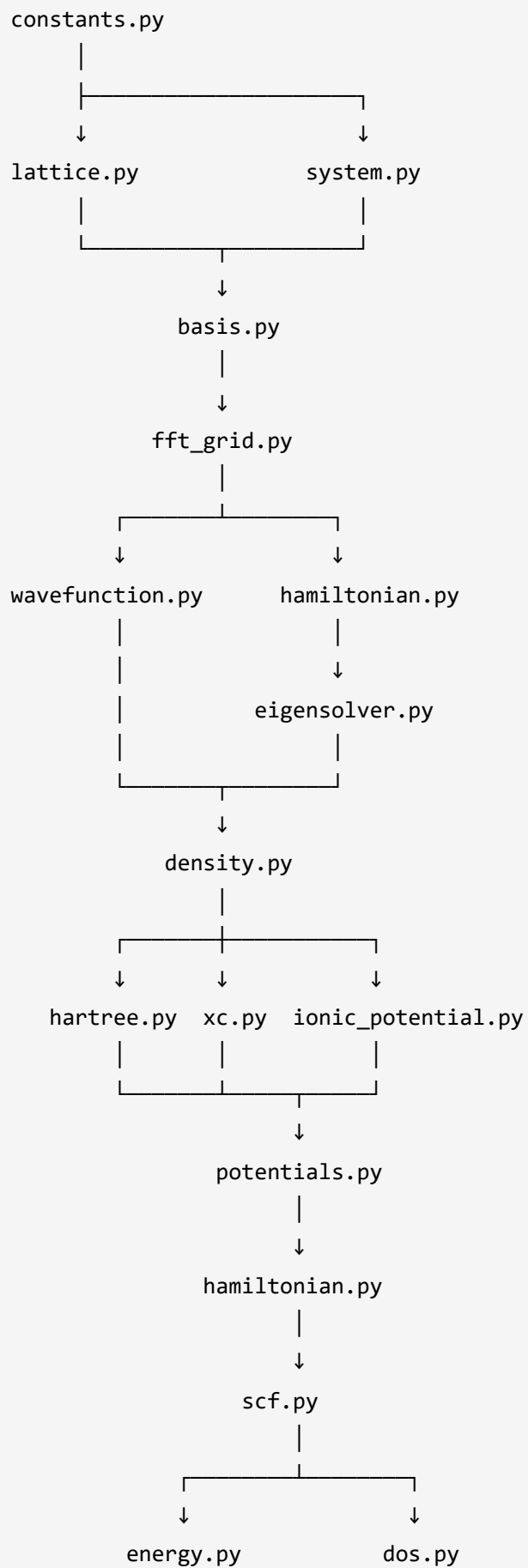
下面这份我把它叫做：

Mini-DFT v0.1 —— 文件级开发说明书

目标是细化到你之后可以真正打开 IDE，按照顺序一个文件一个文件写。

一、先确定整个项目的依赖关系

先看总图，这比直接看代码重要。



这里有一个非常重要的认识：

不是所有文件都是线性执行的。

尤其是：

```
density
  ↓
potential
  ↓
Hamiltonian
  ↓
eigensolver
  ↓
density
```

形成的是一个**循环**。

这就是 SCF。

二、 constants.py

2.1 职责

只负责：

统一单位和物理常数。

建议第一版直接使用：

atomic units

即：

$$\hbar = 1, \quad m_e = 1, \quad e = 1.$$

那么：

$$T = -\frac{1}{2}\nabla^2$$

平面波动能：

$$T_G = \frac{1}{2}|\mathbf{k} + \mathbf{G}|^2$$

Hartree 势：

$$V_H(G) = \frac{4\pi}{G^2} n(G)$$

都会非常干净。

2.2 建议内容

```
import numpy as np

TWO_PI = 2.0 * np.pi

# unit conversion
BOHR_TO_ANGSTROM = ...
HARTREE_TO_EV = ...
EV_TO_HARTREE = ...
```

第一版不要搞复杂的单位系统。

2.3 不负责

不要在这里：

- 定义 System
- 定义晶格
- 计算 G
- 计算势
- 计算 Hamiltonian

三、lattice.py

3.1 职责

解决：

实空间晶格 ↔ 倒空间晶格。

输入：

$$A = \begin{pmatrix} a_{1x} & a_{2x} & a_{3x} & a_{1y} & a_{2y} & a_{3y} & a_{1z} & a_{2z} & a_{3z} \end{pmatrix}.$$

计算：

$$B = 2\pi(A^{-1})^T.$$

3.2 核心对象

建议：

```
class Lattice:
    ...
```

内部保存：

```
A
B
volume
```

也就是：

$$A = (\mathbf{a}_1, \mathbf{a}_2, \mathbf{a}_3)$$

$$B = (\mathbf{b}_1, \mathbf{b}_2, \mathbf{b}_3)$$

3.3 核心函数

```
reciprocal_vectors()
```

返回：

$$\mathbf{b}_1, \mathbf{b}_2, \mathbf{b}_3.$$

```
volume()
```

计算：

$$\Omega = |\det A|.$$

`fractional_to_cartesian()`

$$r_{\text{cart}} = A r_{\text{frac}}.$$

`cartesian_to_fractional()`

$$r_{\text{frac}} = A^{-1} r_{\text{cart}}.$$

3.4 测试

必须检查：

$$\mathbf{a}_i \cdot \mathbf{b}_j = \delta_{ij}$$

这应该成为第一个单元测试。

四、`system.py`

4.1 职责

描述：

我要计算哪个物理体系？

建议定义：

```
class Atom:
    ...

class System:
    ...
```

4.2 System 应该保存什么？

至少：

```
lattice
atoms
electron_number
kpoints
kpoint_weights
encut
```

例如：

```
System(
    lattice=lattice,
    atoms=atoms,
    electron_number=2,
    kpoints=$$$$0, 0, 0$$$$,
    kpoint_weights=$$1.0$$,
    encut=20.0,
)
```

4.3 它不负责

非常重要：

System 不应该计算：

- Hamiltonian
- density
- Hartree
- XC
- SCF

它只是存储体系信息。

五、basis.py

这是第一个真正进入电子结构计算的文件。

5.1 职责

解决：

$$E_{\text{cut}} \rightarrow \mathbf{G}$$

即：

到底有哪些平面波？

5.2 平面波

波函数：

$$\psi_{nk}(r) = \sum_G C_{nk}(G) e^{i(k+G) \cdot r}.$$

对于给定 (k)，保留：

$$\frac{1}{2} |k + G|^2 \leq E_{\text{cut}}$$

的所有 (G)。

5.3 核心对象

```
class PlaneWaveBasis:  
    ...
```

内部至少保存：


```
G_vectors
G_in_cartesian
k_plus_G
kinetic_energy
num_plane_waves
```

即：

$$G_i$$
$$k + G_i$$
$$\frac{1}{2}|k + G_i|^2.$$

5.4 核心函数

```
generate_G_vectors()
```

生成：

$$G = hb_1 + kb_2 + lb_3.$$

```
calculate_kinetic_energies()
```

计算：

$$T_G = \frac{1}{2}|k + G|^2.$$

```
num_plane_waves()
```

返回：

$$N_{\text{PW}}.$$

5.5 一个重要问题

你之前问过：

“平面波是通过 G 截断确定的吗？不能直接给 ENCUT 吗？”

这里正式确定：

用户输入 ENCUT

程序自动生成：

G

和：

N_{PW} .

所以你不用自己指定平面波数量。

六、 `fft_grid.py`

这是为了让：

G -space

和：

r -space

能够高效转换。

6.1 职责

主要解决：

$G \leftrightarrow FFT\ index$

例如：

```
G = (2, -1, 0)
```

究竟对应 FFT 数组：

```
fft_array[i,j,k]
```

中的什么位置？

由这个模块管理。

6.2 核心对象

```
class FFTGrid:  
    ...
```

保存：

```
grid_shape  
G_to_index  
index_to_G
```

6.3 核心操作

```
reciprocal_to_real(C)
```

实现：

$$C(G) \rightarrow \psi(r).$$

以及：

```
real_to_reciprocal(psi)
```

实现：

$$\psi(r) \rightarrow C(G).$$

6.4 为什么不直接写进 `wavefunction.py` ?

因为 FFT grid 本身是一个独立的数值对象。

以后：

- density
- potential
- wavefunction

都需要 FFT。

所以最好让：

```
fft_grid.py
```

成为公共基础设施。

七、 `wavefunction.py`

7.1 职责

管理：

$$C_{nk}(G)$$

以及：

$$\psi_{nk}(r).$$

7.2 核心对象

```
class Wavefunction:  
    ...
```

可以保存：

```
coefficients  
kpoint  
basis
```

其中：

```
coefficients.shape  
=  
(num_bands, num_G)
```

7.3 核心函数

to_real_space()

$$C(G) \rightarrow \psi(r).$$

normalize()

确保：

$$\int |\psi|^2 dr = 1.$$

对于平面波系数也可以进行等价归一化检查。

density()

计算：

$$|\psi(r)|^2.$$

不过这里我建议最终把**体系总密度的计算**放到 `density.py` 。

`wavefunction.py` 主要负责“波函数本身”。

八、 `hamiltonian.py`

这是整个程序的核心文件之一。

8.1 职责

实现：

$$C \rightarrow HC$$

而不是保存：

$$H_{GG'}.$$

8.2 Hamiltonian

$$H = -\frac{1}{2}\nabla^2 + V_{\text{eff}}(r).$$

其中：

$$V_{\text{eff}} = V_{\text{ion}} + V_H + V_{xc}.$$

8.3 `Hamiltonian` 对象

```
class Hamiltonian:
    ...
```

保存：

```
basis  
fft_grid  
Veff(r)
```

8.4 最核心函数

```
apply(c)
```

输入：

$$C(G)$$

输出：

$$HC(G).$$

8.5 内部流程

动能

$$C_G \rightarrow \frac{1}{2}|k + G|^2 C_G.$$

势能

先：

$$C_G \rightarrow \psi(r).$$

然后：

$$V_{\text{eff}}(r)\psi(r).$$

再：

→

最后：

```
\boxed{
HC=TC+VC
}
$$
```

九、eigensolver.py

9.1 职责

解决：

$$HC = CE$$

9.2 第一版

直接调用：

```
scipy.sparse.linalg.eigsh
```

而不是自己写 Davidson。

9.3 最重要的接口

我建议设计成：

```
solve(
    hamiltonian,
    num_bands,
    tolerance,
)
```

其中 `hamiltonian` 提供：


```
hamiltonian.apply(C)
```

9.4 为什么这样设计？

因为以后：

```
eigsolver.py
```

可以从：

```
SciPy eigsh
```

换成：

```
Davidson
```

但：

```
hamiltonian.py
```

完全不用改变。

这就是软件架构上的**解耦**。

十、 density.py

10.1 职责

解决：

$$\psi \rightarrow n$$

公式：

$$n(r) = 2 \sum_{nk} w_k f_{nk} |\psi_{nk}(r)|^2.$$

10.2 核心函数

```
calculate_density(
    wavefunctions,
    occupations,
    kpoint_weights,
)
```

10.3 必须做的检查

计算：

$$\int_{\Omega} n(r), dr.$$

应该得到：

$$N_e$$

如果不是，说明：

- FFT normalization
- wavefunction normalization
- k-point weight
- spin factor

至少有一个出了问题。

十一、 ionic_potential.py

11.1 职责

计算：

$$V_{\text{ion}}(r)$$

11.2 第一版

使用简单的局域周期势。

例如：

$$V_{\text{ion}}(r) = \sum_I V_I(r - R_I).$$

11.3 为什么单独放？

因为以后真正接入：

pseudopotential

甚至：

PAW

时，SCF、density、eigensolver 不需要全部重写。

十二、`hartree.py`

12.1 职责

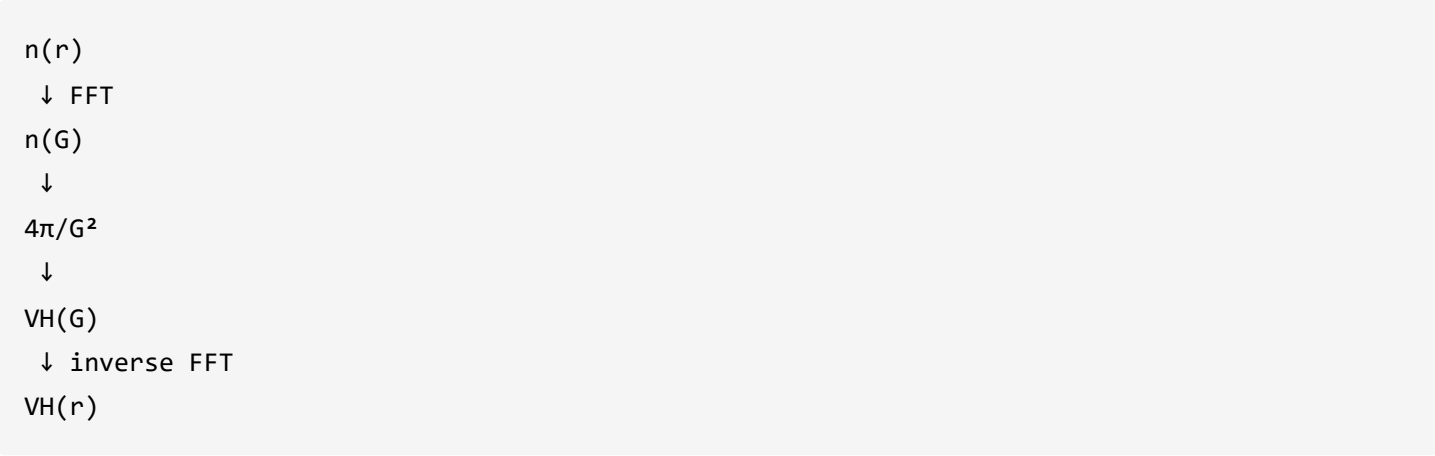
计算：

$$n(r) \rightarrow V_H(r)$$

12.2 核心公式

$$V_H(G) \qquad \qquad \qquad = \frac{4\pi}{G^2} n(G).$$

程序流程：



12.3 特殊情况

：

$$G = 0$$

时：

$$\frac{4\pi}{G^2}$$

发散。

第一版按照原设计：

$V_H(G = 0) = 0$

并把体系限制在适合这种处理的模型。

十三、xc.py

13.1 职责

计算：

$$n(r) \rightarrow V_{xc}(r).$$

第一版：

LDA

13.2 接口

建议：

```
class LDA:
    def potential(self, density):
        ...

    def energy(self, density):
        ...
```

这样以后：

```
LDA
PBE
...
```

都可以实现相同接口。

十四、potentials.py

职责非常简单

把：

$$V_{\text{ion}}$$

$$V_H$$

$$V_{xc}$$

加起来：

$$V_{\text{eff}} = V_{\text{ion}} + V_H + V_{xc}$$

核心函数

```
build_effective_potential(
    Vion,
    VH,
    Vxc,
)
```

它不应该计算 Hartree，也不应该计算 XC。

它只是负责组合。

十五、occupations.py

第一版

不分自旋：

$$2e/\text{orbital}$$

所以：

$$N_{\text{occ}} = N_e/2.$$

核心函数

```
get_occupations(  
    eigenvalues,  
    electron_number,  
)
```

输出：

```
$$1, 1, 0, 0, ...$$
```

这里的 1 表示**空间轨道完全占据**，实际电子数是 2。

十六、 `mixing.py`

第一版：

$$n_{\text{mixed}} == (1 - \alpha)n_{\text{old}} + \alpha n_{\text{new}}.$$

核心函数：

```
mix_density(  
    old_density,  
    new_density,  
    alpha,  
)
```

以后再实现：

```
Pulay  
DIIS  
Kerker
```

十七、 `energy.py`

负责：

$$E_{\text{total}}$$

第一版需要逐步实现：

$$T_s$$

$$E_H$$

$$E_{xc}$$

$$E_{\text{ion}}$$

以及离子-离子项。

这里暂时不要追求“VASP 一模一样”，先保证数学定义和程序内部各项一致。

十八、scf.py

这是整个程序的**总控制器**。

它不应该自己实现物理公式。

它应该只是：

```
初始化 density
  ↓
计算 potentials
  ↓
创建 Hamiltonian
  ↓
调用 eigensolver
  ↓
得到 wavefunctions
  ↓
计算 density_new
  ↓
occupation
  ↓
mixing
  ↓
判断 convergence
  ↓
继续 / 结束
```

核心对象

```
class SCF:
    ...
```

核心函数

```
run()
```

最终：

```
result = scf.run()
```

十九、 dos.py

最后处理：

$$\epsilon_{nk} \rightarrow D(E).$$

公式：

$$D(E) ===== \sum_{nk} w_k \delta(E - \epsilon_{nk}).$$

第一版使用 Gaussian：

$$\delta(E - \epsilon) \approx \frac{1}{\sigma \sqrt{2\pi}} e^{-(E-\epsilon)^2/(2\sigma^2)}.$$

二十、 io.py

这个文件在我刚才回答里漏掉了，但**原设计书里是有的**，所以我们保留它。

职责

负责：

输入文件 / 输出文件。

例如：

POSCAR-like input

读取：

- 晶格
- 原子
- 坐标

输出：

eigenvalues.dat
density.dat
dos.dat

二十一、最后的模块分层

现在我们可以把所有 `.py` 分成五层。

第一层：基础数学

constants.py
lattice.py

↓

第二层：表示体系

```
system.py  
basis.py  
fft_grid.py  
wavefunction.py
```



第三层：电子结构物理

```
density.py  
ionic_potential.py  
hartree.py  
xc.py  
potentials.py
```



第四层：数值求解

```
hamiltonian.py  
eigensolver.py  
occupations.py  
mixing.py
```



第五层：总控制

```
energy.py  
scf.py  
dos.py  
io.py
```

二十二、我们真正开始写代码时的顺序

不要按照文件名从上到下机械地写。

我建议严格：

```
① constants.py
    ↓
② lattice.py
    ↓
③ system.py
    ↓
④ basis.py
    ↓
⑤ fft_grid.py
    ↓
⑥ wavefunction.py
    ↓
⑦ hamiltonian.py
    ↓
⑧ eigensolver.py
    ↓
⑨ density.py
    ↓
⑩ ionic_potential.py
    ↓
⑪ hartree.py
    ↓
⑫ xc.py
    ↓
⑬ potentials.py
    ↓
⑭ occupations.py
    ↓
⑮ mixing.py
    ↓
⑯ energy.py
    ↓
⑰ scf.py
    ↓
⑱ dos.py
    ↓
⑲ io.py
```

但每完成一个，都先测试，再进入下一个。

尤其前 8 个文件，我们甚至可以暂时**完全不涉及 DFT 的自洽问题**，先把：

$$\text{晶格} \rightarrow G \rightarrow \text{平面波} \rightarrow FFT \rightarrow HC \rightarrow \epsilon, C$$

这一整条链打通。

这也是最适合你现在已有量子力学/微扰论基础、但还没有完整 DFT 数值实现经验的路线。